DOCUMENTACIÓN TÉCNICA OFICIAL • ARQUITECTURA DE SOFTWARE

Manual y Especificación Integral de Pruebas de Software

Sistema de Facturación Electrónica UBL 2.1 (SUNAT Perú) con Arquitectura Hexagonal y Suite PHPUnit 10.5

PROYECTO / REPOSITORIO:
Billing (https://github.com/fujimor1/Billing.git)

FECHA DE EMISIÓN:
12 de Noviembre de 2026

VERSIÓN DE PHP / RUNTIME:
PHP 8.2+ / PHP 8.3 • PHPUnit 10.5.66

ESTÁNDAR TRIBUTARIO:
SUNAT UBL 2.1 • XMLDSig SHA-256 • R.S. 183-2004 SPOT

39 / 39	140	0.21s	0
TESTS APROBADOS (100%)	ASERCIONES VERIFICADAS	TIEMPO DE EJECUCIÓN	FALLOS / ERRORES

Este documento constituye la especificación canónica y exhaustiva de todas las suites de pruebas automatizadas del motor de facturación electrónica. Incluye el 100% de los casos de prueba, sus rutas en el repositorio, la regla tributaria/arquitectónica auditada, el código fuente en PHP y las aserciones ejecutadas.

1. Mapa General de Pruebas y Arquitectura

El sistema implementa una arquitectura limpia y desacoplada (Clean Architecture / Hexagonal). La pirámide de pruebas se distribuye en tres niveles rigurosos:

> **Pirámide de Pruebas Implementada:**
> • **Capa Unitaria (26 Tests):** Dominio puro sin efectos secundarios. Valida algoritmos matemáticos (Módulo 11), redondeos simétricos tributarios, cálculo de IGV, detracciones SPOT, y reglas de emisión SUNAT.
> • **Capa de Integración (7 Tests):** Generación DOM XML OASIS UBL 2.1, firma digital XMLDSig con canonización C14N, verificación criptográfica SHA-256, persistencia SQLite ACID e idempotencia contra doble cobro.
> • **Capa Feature / Casos de Uso (6 Tests):** Simulación con Mocks y Spies del ciclo de vida ante el Gateway SOAP de SUNAT, contingencias por timeout, y reversión transaccional (Rollback) del inventario en el Kardex.

1.1 Inventario de Archivos y Rutas Absolutas

Capa	Archivo de Prueba	Ruta en Repositorio	Tests
UNITARIA	**ValueObjectsTest.php**	`backend/tests/Unit/ValueObjectsTest.php`	14 tests
UNITARIA	**CalculadoraTributariaTest.php**	`backend/tests/Unit/CalculadoraTributariaTest.php`	6 tests
UNITARIA	**ComprobanteDomainRulesTest.php**	`backend/tests/Unit/ComprobanteDomainRulesTest.php`	6 tests
INTEGRACIÓN	**XmlAndCryptoIntegrationTest.php**	`backend/tests/Integration/XmlAndCryptoIntegrationTest.php`	4 tests
INTEGRACIÓN	**SqlitePersistenceIntegrationTest.php**	`backend/tests/Integration/SqlitePersistenceIntegrationTest.php`	3 tests
FEATURE	**FacturacionElectronicaMocksFeatureTest.php**	`backend/tests/Feature/FacturacionElectronicaMocksFeatureTest.php`	6 tests
TOTAL SUITE COMPLETA			**39 tests**

1.2 Configuración del Ejecutor (phpunit.xml)

Ubicación: `backend/phpunit.xml`. Configurado según la especificación estricta de PHPUnit 10.5:

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/10.5/phpunit.xsd"
         bootstrap="vendor/autoload.php"
         colors="true"
         stopOnFailure="false">
    <testsuites>
        <testsuite name="Unit">
            <directory>tests/Unit</directory>
        </testsuite>
        <testsuite name="Integration">
            <directory>tests/Integration</directory>
        </testsuite>
        <testsuite name="Feature">
            <directory>tests/Feature</directory>
        </testsuite>
    </testsuites>
    <php>
        <env name="APP_ENV" value="testing"/>
        <env name="DB_CONNECTION" value="sqlite"/>
        <env name="DB_DATABASE" value=":memory:"/>
    </php>
</phpunit>
```

2. Suite Unitaria 1: ValueObjectsTest.php

Ruta: `backend/tests/Unit/ValueObjectsTest.php` • **Ejecución:** 14 tests, 26 aserciones.

Garantiza la inmutabilidad y cumplimiento estricto de los identificadores tributarios peruanos (RUC de 11 dígitos con Módulo 11, DNI de 8 dígitos, Series F001/B001 y Tipos de Cambio).

2.1 Tests 1 al 4: Aceptación de RUCs Válidos con Módulo 11 Oficial

Regla SUNAT: Verifica que la suma ponderada de los primeros 10 dígitos multiplicados por los factores [5, 4, 3, 2, 7, 6, 5, 4, 3, 2], operados bajo el algoritmo módulo 11, resulte exactamente en el dígito verificador. Diferencia RUC 20 (Persona Jurídica) de RUC 10 (Persona Natural).

```
#[DataProvider('proveedorRucsValidos')]
#[TestDox('Acepta números de RUC válidos verificando el algoritmo oficial Módulo 11 de SUNAT')]
public function testAceptaRucsValidosConAlgoritmoModulo11(string $numeroRuc, bool $esJuridica): void
{
    $ruc = new Ruc($numeroRuc);

    $this->assertSame(trim($numeroRuc), $ruc->valor());
    $this->assertSame($esJuridica, $ruc->esPersonaJuridica());
    $this->assertSame(!$esJuridica, $ruc->esPersonaNatural());
}

public static function proveedorRucsValidos(): array
{
    return [
        'SUNAT (Persona Jurídica)' => ['20131312955', true],
        'Banco de Crédito BCP (Persona Jurídica)' => ['20100047218', true],
        'Alicorp SAA (Persona Jurídica)' => ['20100055237', true],
        'Contribuyente Persona Natural con Negocio' => ['10456789124', false],
    ];
}
```

2.2 Tests 5 al 10: Rechazo de RUCs con Formato Inválido o Verificador Adulterado

Regla SUNAT: Lanza `DocumentoIdentidadInvalidoException` si el RUC tiene menos o más de 11 dígitos, letras, prefijos inexistentes (como 99) o dígito verificador calculado erróneo.

```
#[DataProvider('proveedorRucsInvalidos')]
#[TestDox('Rechaza números de RUC con longitud incorrecta, prefijo no autorizado o dígito verificador alterado')]
public function testRechazaRucsInvalidos(string $rucInvalido, string $mensajeEsperado): void
{
    $this->expectException(DocumentoIdentidadInvalidoException::class);
    $this->expectExceptionMessage($mensajeEsperado);

    new Ruc($rucInvalido);
}

public static function proveedorRucsInvalidos(): array
{
    return [
        'Menos de 11 dígitos' => ['2013131295', '11 dígitos'],
        'Más de 11 dígitos' => ['201313129559', '11 dígitos'],
        'Contiene letras' => ['2013131295A', '11 dígitos'],
        'Prefijo 99 no existente en SUNAT' => ['99131312955', 'prefijo inválido'],
        'Dígito verificador Módulo 11 adulterado (termina en 4 en vez de 5)' => ['20131312954', 'dígito verificador'],
        'Dígito verificador Módulo 11 adulterado en RUC 10' => ['10456789120', 'dígito verificador'],
    ];
}
```

2.3 Test 11: Validación Estricta de DNI

Regla RENIEC / SUNAT: Requiere 8 caracteres numéricos exactos. Prohíbe números nulos o puros ceros (00000000).

```
#[TestDox('Valida DNI de 8 dígitos y rechaza DNI de puros ceros o alfanumérico')]
public function testValidacionDeDni(): void
{
    $dni = new Dni('72345678');
    $this->assertSame('72345678', $dni->valor());

    $this->expectException(DocumentoIdentidadInvalidoException::class);
    new Dni('00000000');
}
```

2.4 Tests 12 al 14: Formateo de Series, Validación Tipo Comprobante y Moneda Extranjera

```
#[TestDox('Formatea correctamente Serie y Correlativo a 8 dígitos (F001-00000045)')]
public function testFormateoDeSerieYCorrelativo(): void
{
    $serie = new SerieCorrelativo('f001', 45, TipoComprobante::FACTURA);

    $this->assertSame('F001', $serie->serie());
    $this->assertSame(45, $serie->correlativo());
    $this->assertSame('00000045', $serie->correlativoFormateado());
    $this->assertSame('F001-00000045', $serie->formatoCompleto());
}

#[TestDox('Impide usar una serie de Boleta (B001) en una Factura Electrónica (01)')]
public function testRechazaIncoherenciaEntreSerieYTipoComprobante(): void
{
    $this->expectException(ReglaTributariaException::class);
    $this->expectExceptionMessage("Una Factura Electrónica (01) debe tener una serie que inicie con 'F'");

    new SerieCorrelativo('B001', 1, TipoComprobante::FACTURA);
}

#[TestDox('Exige un tipo de cambio válido mayor a 1.00 cuando la moneda es Dólares (USD)')]
public function testMonedaExtranjeraExigeTipoDeCambioValido(): void
{
    $usd = new Moneda(Moneda::USD, 3.754);
    $this->assertTrue($usd->esExtranjera());
    $this->assertSame(375.40, $usd->convertirASoles(100.00));

    $this->expectException(ReglaTributariaException::class);
    new Moneda(Moneda::USD, 1.0);
}
```

3. Suite Unitaria 2: CalculadoraTributariaTest.php

Ruta: `backend/tests/Unit/CalculadoraTributariaTest.php` • **Ejecución:** 6 tests, 28 aserciones.

Comprueba la exactitud del cálculo de base imponible, IGV (18%), operaciones exoneradas (20), inafectas (30), bolsas plásticas (ICBPER), transferencias gratuitas y el sistema de detracciones SPOT.

3.1 Test 15: Operación Gravada Estándar

Valida que el cálculo de productos comerciales tradicionales aplique el 18% exacto sobre la sumatoria neta.

```
#[TestDox('Calcula exactamente una operación gravada estándar con IGV 18%')]
public function testCalculaOperacionGravadaEstandar(): void
{
    $items = [
        new ItemComprobante('LAP-01', 'Laptop Empresarial Core i7', 2, 2500.00, TipoAfectacionIgv::GRAVADO_OPERACION_ONEROSA),
        new ItemComprobante('MOU-01', 'Mouse Inalámbrico', 5, 50.00, TipoAfectacionIgv::GRAVADO_OPERACION_ONEROSA),
    ];

    $resumen = $this->calculadora->calcular($items);

    // 2 * 2500 + 5 * 50 = 5250.00
    $this->assertSame(5250.00, $resumen->totalGravadas);
    $this->assertSame(945.00, $resumen->totalIgv); // 18% de 5250
    $this->assertSame(6195.00, $resumen->importeTotal);
    $this->assertSame(0.0, $resumen->totalExoneradas);
    $this->assertSame(0.0, $resumen->totalInafectas);
}
```

3.2 Test 16: Comprobante Mixto Complejo (Gravado + Exonerado + Gratuito + ICBPER)

Valida la convivencia de múltiples tipos de afectación en una sola factura sin contaminación entre bolsas fiscales.

```
#[TestDox('Calcula factura mixta: Gravados + Exonerados (Amazonía/Libros) + Inafectos + Bonificación Gratuita + Bolsas ICBPER')]
public function testCalculaComprobanteMixtoComplejo(): void
{
    $items = [
        new ItemComprobante('PROD-GRAV', 'Monitor 27"', 10, 100.00, TipoAfectacionIgv::GRAVADO_OPERACION_ONEROSA, descuentoPorcen
taje: 10.0),
        new ItemComprobante('LIB-EXO', 'Libro Testing', 2, 150.00, TipoAfectacionIgv::EXONERADO_OPERACION_ONEROSA),
        new ItemComprobante('PREMIO-01', 'Teclado Bonificado', 1, 200.00, TipoAfectacionIgv::GRAVADO_RETIRO_PREMIO),
        new ItemComprobante('BOLSA-BIO', 'Bolsa Plástica', 4, 0.10, TipoAfectacionIgv::GRAVADO_OPERACION_ONEROSA, aplicaIcbper: t
rue),
    ];

    $resumen = $this->calculadora->calcular($items);

    $this->assertSame(900.40, $resumen->totalGravadas);
    $this->assertSame(300.00, $resumen->totalExoneradas);
    $this->assertSame(200.00, $resumen->totalGratuitas);
    $this->assertSame(36.00, $resumen->totalIgvGratuitas);
    $this->assertSame(100.00, $resumen->totalDescuentosItems);
    $this->assertSame(162.07, $resumen->totalIgv);
    $this->assertSame(2.00, $resumen->totalIcbper);
    $this->assertSame(164.07, $resumen->totalImpuestos);
    $this->assertSame(1200.40, $resumen->valorVentaNeto);
    $this->assertSame(1364.47, $resumen->importeTotal);
}
```

3.3 Tests 17 y 18: Régimen de Detracciones SPOT (R.S. 183-2004)

Valida la aplicación automática del 12% sobre servicios que superen S/ 700.00 (incluso facturados en USD) y su exención si no superan el tope.

```
#[TestDox('Aplica Detracción SPOT (12%) automáticamente cuando el servicio supera S/ 700.00 o equivalente en USD')]
public function testCalculaDetraccionSpotEnDolaresCuandoSuperaTopeEnSoles(): void
{
    $items = [new ItemComprobante('SERV-TI', 'Auditoría QA', 1, 300.00, TipoAfectacionIgv::GRAVADO_OPERACION_ONEROSA, unidadMedid
a: 'ZZ')];
    $monedaUsd = new Moneda(Moneda::USD, 3.80);
    $resumen = $this->calculadora->calcular($items, $monedaUsd, 0.0, 12.0);

    $this->assertSame(354.00, $resumen->importeTotal);
    $this->assertSame(42.48, $resumen->montoDetraccion);
    $this->assertSame(311.52, $resumen->montoNetoPendientePago);
}

#[TestDox('No aplica Detracción SPOT si el importe total en Soles no supera los S/ 700.00')]
public function testNoAplicaDetraccionSiMontoNoSuperaSetecientosSoles(): void
{
    $items = [new ItemComprobante('SERV-MANT', 'Soporte Técnico', 1, 500.00, TipoAfectacionIgv::GRAVADO_OPERACION_ONEROSA, unidad
Medida: 'ZZ')];
    $resumen = $this->calculadora->calcular($items, new Moneda(Moneda::PEN), 0.0, 12.0);

    $this->assertSame(590.00, $resumen->importeTotal);
    $this->assertSame(0.0, $resumen->montoDetraccion);
    $this->assertSame(590.00, $resumen->montoNetoPendientePago);
}
```

3.4 Tests 19 y 20: Descuento en Gratuitos y Operaciones Inafectas Estrictas (30)

```
#[TestDox('Lanza excepción si se intenta aplicar descuento comercial a una bonificación gratuita')]
public function testImpideDescuentoEnItemGratuito(): void
{
    $this->expectException(ReglaTributariaException::class);
    $this->expectExceptionMessage('No se puede aplicar descuento comercial a una transferencia gratuita');

    new ItemComprobante('BONO-01', 'Muestra Médica', 1, 50.00, TipoAfectacionIgv::INAFECTO_BONIFICACION, descuentoPorcentaje: 15.
0);
}

#[TestDox('Calcula exactamente una operación inafecta al IGV (30) con base imponible y 0% de IGV')]
public function testCalculaOperacionInafectaEstricta(): void
{
    $items = [new ItemComprobante('CURS-EDU', 'Diplomado Software', 1, 1500.00, TipoAfectacionIgv::INAFECTO_OPERACION_ONEROSA, un
idadMedida: 'ZZ')];
    $resumen = $this->calculadora->calcular($items);

    $this->assertSame(1500.00, $resumen->totalInafectas);
    $this->assertSame(0.0, $resumen->totalGravadas);
    $this->assertSame(0.0, $resumen->totalIgv);
    $this->assertSame(1500.00, $resumen->importeTotal);
    $this->assertSame(1500.00, $resumen->valorVentaNeto);
}
```

4. Suite Unitaria 3: ComprobanteDomainRulesTest.php

Ruta: `backend/tests/Unit/ComprobanteDomainRulesTest.php` • **Ejecución:** 6 tests, 11 aserciones.

Audita las restricciones fiscales de emisión (Reglas SUNAT 2017 y 2511), el saldo vivo de las facturas ante Notas de Crédito, y el plazo perentorio de 7 días para Comunicaciones de Baja.

4.1 Tests 21 al 23: Reglas Fiscales de Emisión SUNAT 2017 y 2511

Regla 2017: Una Factura jamás puede emitirse a DNI. **Regla 2511:** Una Boleta mayor a S/ 700.00 no puede emitirse a "Clientes Varios".

```
#[TestDox('Regla SUNAT 2017: Impide emitir una Factura Electrónica (01) a un cliente con DNI')]
public function testImpideEmitirFacturaAClienteConDni(): void
{
    $clienteDni = Cliente::conDni('72345678', 'Juan Pérez');
    $items = [new ItemComprobante('P01', 'Teclado', 1, 100.00)];

    $this->expectException(ReglaTributariaException::class);
    $this->expectExceptionMessage('Error 2017');

    new Comprobante(new SerieCorrelativo('F001', 1, TipoComprobante::FACTURA), TipoComprobante::FACTURA, $clienteDni, $items, $th
is->calculadora->calcular($items));
}

#[TestDox('Regla SUNAT 2511: Impide emitir Boleta mayor a S/ 700.00 a Clientes Varios (Sin DNI)')]
public function testImpideBoletaMayorASetecientosSolesSinIdentificarCliente(): void
{
    $items = [new ItemComprobante('TV-01', 'Smart TV 50"', 1, 650.00)]; // Total con IGV: S/ 767.00 (> 700)

    $this->expectException(ReglaTributariaException::class);
    $this->expectExceptionMessage('Error 2511');

    new Comprobante(new SerieCorrelativo('B001', 1, TipoComprobante::BOLETA), TipoComprobante::BOLETA, Cliente::clienteVarios(),
$items, $this->calculadora->calcular($items));
}

#[TestDox('Permite emitir Boleta menor o igual a S/ 700.00 a Clientes Varios')]
public function testPermiteBoletaMenorASetecientosSolesAClientesVarios(): void
{
    $items = [new ItemComprobante('AUD-01', 'Audífonos Bluetooth', 1, 100.00)];
    $boleta = new Comprobante(new SerieCorrelativo('B001', 10, TipoComprobante::BOLETA), TipoComprobante::BOLETA, Cliente::client
eVarios(), $items, $this->calculadora->calcular($items));

    $this->assertSame('B001-00000010', $boleta->numeroCompleto());
    $this->assertSame(EstadoComprobante::BORRADOR, $boleta->estado());
}
```

4.2 Test 24: Control de Saldo Vivo de Factura y Prevención de Sobregiro en NC

Valida que las Notas de Crédito parciales descuenten el saldo de la factura original y rechacen devoluciones que excedan dicho saldo.

```
#[TestDox('Controla el saldo de una Factura ante Notas de Crédito parciales e impide devolver más del total')]
public function testControlaSaldoDeFacturaConNotasDeCreditoParciales(): void
{
    $cliente = Cliente::conRuc('20131312955', 'EMPRESA DEMO SAC');
    $itemsFactura = [new ItemComprobante('LAP-01', 'Laptop', 2, 1000.00)];
    $factura = new Comprobante(new SerieCorrelativo('F001', 100, TipoComprobante::FACTURA), TipoComprobante::FACTURA, $cliente,
$itemsFactura, $this->calculadora->calcular($itemsFactura));
    $factura->procesarRespuestaSunat('0', 'Aceptada');

    // NC1 por 1 Laptop (S/ 1,180.00)
    $itemsNc1 = [new ItemComprobante('LAP-01', 'Laptop', 1, 1000.00)];
    $nc1 = Comprobante::crearNotaCredito(new SerieCorrelativo('FC01', 1, TipoComprobante::NOTA_CREDITO), $factura, MotivoNotaCred
ito::DEVOLUCION_POR_ITEM, 'Devolución 1 equipo', $itemsNc1, $this->calculadora->calcular($itemsNc1));
    $factura->aplicarNotaCreditoAceptada($nc1->resumenImpuestos()->importeTotal, MotivoNotaCredito::DEVOLUCION_POR_ITEM);

    $this->assertSame(1180.00, $factura->saldoDisponibleParaNotasCredito());

    // NC2 por 2 Laptops (S/ 2,360.00 > S/ 1,180.00 disponible) DEBE FALLAR
    $this->expectException(ReglaTributariaException::class);
    $this->expectExceptionMessage('excede el saldo disponible');

    Comprobante::crearNotaCredito(new SerieCorrelativo('FC01', 2, TipoComprobante::NOTA_CREDITO), $factura, MotivoNotaCredito::DE
VOLUCION_POR_ITEM, 'Devolución excedida', $itemsFactura, $this->calculadora->calcular($itemsFactura));
}
```

4.3 Tests 25 y 26: Plazo Legal de 7 Días para Comunicaciones de Baja (RA)

SUNAT exige que las anulaciones por Comunicación de Baja se envíen dentro de 7 días naturales posteriores a la emisión.

```
#[TestDox('Impide anular por Comunicación de Baja una Factura emitida hace más de 7 días calendario')]
public function testImpideComunicacionDeBajaPasadosSieteDias(): void
{
    $fechaEmision = new DateTimeImmutable('2026-11-01 10:00:00');
    $fechaBajaExtemporanea = new DateTimeImmutable('2026-11-10 10:00:00'); // 9 días después

    $factura = new Comprobante(new SerieCorrelativo('F001', 50, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::con
Ruc('20100047218', 'BANCO DE CREDITO'), [new ItemComprobante('SERV-01', 'Consultoría', 1, 1000.00)], $this->calculadora->calcular
([new ItemComprobante('SERV-01', 'Consultoría', 1, 1000.00)]), new Moneda(Moneda::PEN), $fechaEmision);
    $factura->procesarRespuestaSunat('0', 'Aceptado');

    $this->expectException(ReglaTributariaException::class);
    $this->expectExceptionMessage('El plazo máximo para enviar una Comunicación de Baja es de 7 días');

    $factura->anularPorComunicacionBaja('Error en emisión', $fechaBajaExtemporanea);
}

#[TestDox('Permite Comunicación de Baja dentro de los 7 días y cambia estado a ANULADO_BAJA')]
public function testPermiteComunicacionDeBajaDentroDelPlazo(): void
{
    $fechaEmision = new DateTimeImmutable('2026-11-01 10:00:00');
    $fechaBajaValida = new DateTimeImmutable('2026-11-05 15:00:00'); // 4 días después

    $factura = new Comprobante(new SerieCorrelativo('F001', 51, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::con
Ruc('20100047218', 'BANCO DE CREDITO'), [new ItemComprobante('SERV-01', 'Consultoría', 1, 1000.00)], $this->calculadora->calcular
([new ItemComprobante('SERV-01', 'Consultoría', 1, 1000.00)]), new Moneda(Moneda::PEN), $fechaEmision);
    $factura->procesarRespuestaSunat('0', 'Aceptado');

    $factura->anularPorComunicacionBaja('Cliente desistió', $fechaBajaValida);
    $this->assertSame(EstadoComprobante::ANULADO_BAJA, $factura->estado());
}
```


Página 8 de 16 • Facturación Electrónica UBL 2.1 SUNAT • Suite PHPUnit 10.5 (39 Tests / 140 Assertions)

5. Suite de Integración 1: XmlAndCryptoIntegrationTest.php

Ruta: `backend/tests/Integration/XmlAndCryptoIntegrationTest.php` • **Ejecución:** 4 tests, 20 aserciones.

Verifica la construcción del árbol DOM según el estándar OASIS UBL 2.1, la firma digital XMLDSig SHA-256, la detección de adulteraciones y el esquema fiscal para operaciones inafectas.

5.1 Test 27: Estructura DOM UBL 2.1 y Nodos Obligatorios con XPath

```
#[TestDox('Genera un documento XML UBL 2.1 válido y verifica sus nodos obligatorios mediante consultas XPath')]
public function testGeneraEstructuraXmlUbl21ValidaConXPath(): void
{
    $items = [new ItemComprobante('SRV-CLOUD', 'Servidor Cloud', 2, 1000.00)];
    $factura = new Comprobante(new SerieCorrelativo('F001', 25, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::con
Ruc('20100047218', 'BANCO DE CREDITO DEL PERU'), $items, $this->calculadora->calcular($items));

    $xmlString = $this->xmlGenerator->generarXmlUbl21($factura);
    $dom = new DOMDocument();
    $this->assertTrue($dom->loadXML($xmlString));

    $xpath = new DOMXPath($dom);
    $xpath->registerNamespace('cbc', 'urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2');
    $xpath->registerNamespace('cac', 'urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2');

    $this->assertSame('2.1', $xpath->evaluate('string(//cbc:UBLVersionID)'));
    $this->assertSame('F001-00000025', $xpath->evaluate('string(/child::*/cbc:ID)'));
    $this->assertSame('01', $xpath->evaluate('string(//cbc:InvoiceTypeCode)'));
    $this->assertSame('20100047218', $xpath->evaluate('string(//cac:AccountingCustomerParty//cbc:ID)'));
    $this->assertSame('360.00', $xpath->evaluate('string(//cac:TaxTotal/cbc:TaxAmount)'));
    $this->assertSame('2360.00', $xpath->evaluate('string(//cac:LegalMonetaryTotal/cbc:PayableAmount)'));
}
```

5.2 Test 28: Firma Criptográfica SHA-256 e Integridad XMLDSig

Valida la inyección canónica de los elementos `ds:DigestValue` y `ds:SignatureValue`.

```
#[TestDox('Firma digitalmente el XML UBL 2.1 con SHA-256 e inyecta DigestValue y SignatureValue verificables')]
public function testFirmaCriptograficamenteElXmlYVerificaSuIntegridad(): void
{
    $items = [new ItemComprobante('LAP-01', 'Laptop Gamer', 1, 3000.00)];
    $factura = new Comprobante(new SerieCorrelativo('F001', 7, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::conR
uc('20100055237', 'ALICORP SAA'), $items, $this->calculadora->calcular($items));

    $xmlSinFirmar = $this->xmlGenerator->generarXmlUbl21($factura);
    $resultadoFirma = $this->digitalSigner->firmarXml($xmlSinFirmar);

    $this->assertNotEmpty($resultadoFirma->hashCpe);
    $this->assertNotEmpty($resultadoFirma->signatureValue);
    $this->assertStringContainsString('<ds:DigestValue>', $resultadoFirma->xmlFirmado);
    $this->assertStringContainsString('<ds:SignatureValue>', $resultadoFirma->xmlFirmado);
    $this->assertTrue($this->digitalSigner->verificarIntegridadXml($resultadoFirma->xmlFirmado));
}
```

5.3 Test 29: Detección de Ataque por Adulteración de Importe y Rechazo SUNAT 2335

Demuestra que si un atacante altera un valor monetario en el XML ya firmado, la verificación de integridad falla y el gateway de SUNAT emite el código de rechazo 2335.

```
#[TestDox('Detecta adulteración (ataque de modificación de importe en el XML firmado) y SUNAT lo rechaza con código 2335')]
public function testDetectaAdulteracionDeImporteEnXmlFirmadoYSunatLoRechaza(): void
{
    $items = [new ItemComprobante('LAP-01', 'Laptop Corporativa', 1, 1000.00)];
    $factura = new Comprobante(new SerieCorrelativo('F001', 99, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::con
Ruc('20100055237', 'ALICORP SAA'), $items, $this->calculadora->calcular($items));

    $xmlSinFirmar = $this->xmlGenerator->generarXmlUbl21($factura);
    $resultadoFirma = $this->digitalSigner->firmarXml($xmlSinFirmar);

    // Ataque malicioso: Se modifica el monto firmado de 1180.00 a 10.00
    $xmlAdulterado = str_replace('1180.00', '10.00', $resultadoFirma->xmlFirmado);

    $this->assertFalse($this->digitalSigner->verificarIntegridadXml($xmlAdulterado));

    $gateway = new SunatSoapSimulatorGateway($this->digitalSigner);
    $cdr = $gateway->enviarComprobante($factura, $xmlAdulterado);

    $this->assertSame('2335', $cdr->codigo);
    $this->assertStringContainsString('alterado o la firma digital es inválida', $cdr->mensaje);
}
```

5.4 Test 30: Esquema XML UBL 2.1 para Inafectos (9998 INA / FRE)

Valida la correspondencia tributaria internacional en el nodo `cac:TaxScheme`.

```
#[TestDox('Genera XML UBL 2.1 con operaciones inafectas (30) asociadas al esquema tributario SUNAT 9998 INA / FRE')]
public function testGeneraXmlUbl21ConOperacionesInafectasConEsquema9998(): void
{
    $items = [new ItemComprobante('CURS-EDU-INAF', 'Diplomado Software', 1, 1500.00, \App\Domain\Enums\TipoAfectacionIgv::INAFECT
O_OPERACION_ONEROSA, unidadMedida: 'ZZ')];
    $factura = new Comprobante(new SerieCorrelativo('F001', 30, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::con
Ruc('20100047218', 'BANCO DE CREDITO'), $items, $this->calculadora->calcular($items));

    $xmlString = $this->xmlGenerator->generarXmlUbl21($factura);
    $dom = new DOMDocument();
    $this->assertTrue($dom->loadXML($xmlString));

    $xpath = new DOMXPath($dom);
    $xpath->registerNamespace('cbc', 'urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2');
    $xpath->registerNamespace('cac', 'urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2');

    $this->assertSame('9998', $xpath->evaluate('string(//cac:TaxCategory/cac:TaxScheme/cbc:ID)'));
    $this->assertSame('INA', $xpath->evaluate('string(//cac:TaxCategory/cac:TaxScheme/cbc:Name)'));
    $this->assertSame('FRE', $xpath->evaluate('string(//cac:TaxCategory/cac:TaxScheme/cbc:TaxTypeCode)'));
    $this->assertSame('0.00', $xpath->evaluate('string(//cac:TaxSubtotal/cbc:TaxAmount)'));
    $this->assertSame('1500.00', $xpath->evaluate('string(//cac:TaxSubtotal/cbc:TaxableAmount)'));
}
```

6. Suite de Integración 2: SqlitePersistenceIntegrationTest.php

Ruta: `backend/tests/Integration/SqlitePersistenceIntegrationTest.php` • **Ejecución:** 3 tests, 15 aserciones.

Audita la base de datos relacional SQLite en memoria, el aislamiento secuencial de correlativos por serie y la máquina de estados de la tabla de idempotencia.

6.1 Test 31: Catálogo Inicial Completo (9 Productos) y Actualización de Stock

```
#[TestDox('Sembrado inicial carga el catálogo de productos y permite actualizar el stock en SQLite')]
public function testCatalogoInicialYActualizacionDeStockEnSqlite(): void
{
    $productos = $this->productoRepo->listarTodos();
    $this->assertCount(9, $productos);

    $laptop = $this->productoRepo->buscarPorCodigo('LAP-PRO-16');
    $this->assertNotNull($laptop);
    $this->assertSame(15, $laptop->stockActual());

    $laptop->descontarStock(3);
    $this->productoRepo->guardar($laptop);

    $laptopActualizada = $this->productoRepo->buscarPorCodigo('LAP-PRO-16');
    $this->assertSame(12, $laptopActualizada?->stockActual());
}
```

6.2 Test 32: Correlativo Secuencial Aislado por Serie Fiscal

Garantiza que la emisión de una Factura (F001) avance su propio correlativo a 2 sin alterar el correlativo de Boletas (B001).

```
#[TestDox('Persiste y recupera un Comprobante completo manteniendo su correlativo secuencial por serie')]
public function testPersistenciaYCorrelativoSecuencialPorSerie(): void
{
    $this->assertSame(1, $this->comprobanteRepo->siguienteCorrelativo('F001'));
    $this->assertSame(1, $this->comprobanteRepo->siguienteCorrelativo('B001'));

    $items = [new ItemComprobante('LAP-PRO-16', 'Laptop Workstation', 1, 3500.00)];
    $factura1 = new Comprobante(new SerieCorrelativo('F001', 1, TipoComprobante::FACTURA), TipoComprobante::FACTURA, Cliente::con
Ruc('20131312955', 'SUNAT'), $items, $this->calculadora->calcular($items));
    $factura1->registrarFirmaDigital('<xml>firmado</xml>', 'HASH123456=');
    $factura1->procesarRespuestaSunat('0', 'Aceptada');

    $this->comprobanteRepo->guardar($factura1);

    $this->assertSame(2, $this->comprobanteRepo->siguienteCorrelativo('F001'));
    $this->assertSame(1, $this->comprobanteRepo->siguienteCorrelativo('B001'));

    $recuperada = $this->comprobanteRepo->buscarPorNumero('F001-00000001');
    $this->assertNotNull($recuperada);
    $this->assertSame(EstadoComprobante::ACEPTADO, $recuperada->estado());
    $this->assertSame('HASH123456=', $recuperada->hashCpe());
    $this->assertSame(4130.00, $recuperada->resumenImpuestos()->importeTotal);
}
```

6.3 Test 33: Tabla de Idempotencia contra Doble Clic y Repetición de Peticiones

Valida la transición segura `IN_PROGRESS` → `COMPLETED` guardando la respuesta JSON serializada.

```
#[TestDox('La tabla de Idempotencia registra transacciones IN_PROGRESS y recupera la respuesta COMPLETED')]
public function testTablaIdempotenciaPrevieneDobleEmision(): void
{
    $this->pdo->exec("CREATE TABLE IF NOT EXISTS idempotency_keys (
        key TEXT PRIMARY KEY,
        status TEXT NOT NULL,
        response_json TEXT,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )");

    $key = 'idem-unit-test-123';

    // 1. Inserción inicial en proceso
    $stmt = $this->pdo->prepare('INSERT INTO idempotency_keys (key, status, response_json) VALUES (:k, :s, :r)');
    $stmt->execute([':k' => $key, ':s' => 'IN_PROGRESS', ':r' => null]);

    $check = $this->pdo->prepare('SELECT status, response_json FROM idempotency_keys WHERE key = :k');
    $check->execute([':k' => $key]);
    $row = $check->fetch(\PDO::FETCH_ASSOC);
    $this->assertSame('IN_PROGRESS', $row['status']);
    $this->assertNull($row['response_json']);

    // 2. Transición a COMPLETED con la respuesta serializada
    $dummyResponse = json_encode(['ok' => true, 'numero' => 'F001-00000001']);
    $update = $this->pdo->prepare('UPDATE idempotency_keys SET status = :s, response_json = :r WHERE key = :k');
    $update->execute([':s' => 'COMPLETED', ':r' => $dummyResponse, ':k' => $key]);

    $check->execute([':k' => $key]);
    $rowCompleted = $check->fetch(\PDO::FETCH_ASSOC);
    $this->assertSame('COMPLETED', $rowCompleted['status']);
    $this->assertSame($dummyResponse, $rowCompleted['response_json']);
}
```


Página 12 de 16 • Facturación Electrónica UBL 2.1 SUNAT • Suite PHPUnit 10.5 (39 Tests / 140 Assertions)

7. Suite Feature: FacturacionElectronicaMocksFeatureTest.php

Ruta: `backend/tests/Feature/FacturacionElectronicaMocksFeatureTest.php` • **Ejecución:** 6 tests, 20 aserciones.

Comprueba los casos de uso principales utilizando dobles de prueba (Mocks y Spies) para simular la conducta de SUNAT, contingencias por desconexión y rollback atómico del stock en el Kardex.

7.1 Test 34: Caso 1 - Factura Aceptada por SUNAT y Descuento en Kardex

```
#[TestDox('Caso 1 (Mock Aceptado): Emite Factura, SUNAT devuelve CDR código 0 y se descuenta el stock en el Kardex')]
public function testEmiteFacturaAceptadaPorSunatYDescuentaStockEnKardex(): void
{
    $sunatMock = $this->createMock(SunatGatewayInterface::class);
    $sunatMock->expects($this->once())
        ->method('enviarComprobante')
        ->willReturn(new RespuestaSunatCdr('0', 'La Factura F001-00000001 ha sido aceptada', '<cdr/>'));

    $useCase = new EmitirComprobanteUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xmlGenerator,
$this->digitalSigner, $sunatMock);

    $comprobante = $useCase->ejecutar(serie: 'F001', tipo: TipoComprobante::FACTURA, cliente: Cliente::conRuc('20100047218', 'BAN
CO DE CREDITO DEL PERU'), items: [new ItemComprobante('LAP-PRO-16', 'Laptop Workstation Pro 16"', 5, 3500.00)]);

    $this->assertSame('F001-00000001', $comprobante->numeroCompleto());
    $this->assertSame(EstadoComprobante::ACEPTADO, $comprobante->estado());
    $this->assertSame(10, $this->productoRepo->buscarPorCodigo('LAP-PRO-16')?->stockActual()); // Stock baja de 15 a 10
}
```

7.2 Test 35: Caso 2 - Caída de Red / Timeout hacia Modo Contingencia

Garantiza que una falla de conexión SOAP con SUNAT no interrumpa la venta, marcándola en CONTINGENCIA y reservando el stock.

```
#[TestDox('Caso 2 (Mock Caída de Red / Timeout): Si SUNAT falla por red, la factura pasa a CONTINGENCIA sin perder la venta')]
public function testPasaAContingenciaCuandoElServidorDeSunatSufreCaidaDeRed(): void
{
    $sunatMock = $this->createMock(SunatGatewayInterface::class);
    $sunatMock->expects($this->once())
        ->method('enviarComprobante')
        ->willThrowException(new RuntimeException('SOAP Fault: Connection timed out after 30000ms'));

    $useCase = new EmitirComprobanteUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xmlGenerator,
$this->digitalSigner, $sunatMock);

    $comprobante = $useCase->ejecutar(serie: 'F001', tipo: TipoComprobante::FACTURA, cliente: Cliente::conRuc('20100055237', 'ALI
CORP SAA'), items: [new ItemComprobante('MON-4K-27', 'Monitor IPS 27"', 2, 1100.00)]);

    $this->assertSame(EstadoComprobante::CONTINGENCIA, $comprobante->estado());
    $this->assertSame('NETWORK_TIMEOUT', $comprobante->codigoRespuestaSunat());
    $this->assertSame(23, $this->productoRepo->buscarPorCodigo('MON-4K-27')?->stockActual());
}
```

7.3 Test 36: Caso 3 - Rollback Automático de Stock ante Rechazo SUNAT (Código 2800)

Si SUNAT rechaza la factura por inconsistencia fiscal del receptor, el sistema cancela la operación y repone el stock automáticamente.

```
#[TestDox('Caso 3 (Mock Rechazo Fiscal SUNAT): Si SUNAT rechaza la factura (código 2800), hace Rollback automático del Stock')]
public function testRevierteElStockAutomaticamenteCuandoSunatRechazaElComprobante(): void
{
    $sunatMock = $this->createMock(SunatGatewayInterface::class);
    $sunatMock->expects($this->once())
        ->method('enviarComprobante')
        ->willReturn(new RespuestaSunatCdr('2800', 'Rechazo: Contribuyente en Baja de Oficio'));

    $useCase = new EmitirComprobanteUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xmlGenerator,
$this->digitalSigner, $sunatMock);

    $comprobante = $useCase->ejecutar(serie: 'F001', tipo: TipoComprobante::FACTURA, cliente: Cliente::conRuc('20131312955', 'EMP
RESA CON BAJA DE OFICIO SAC'), items: [new ItemComprobante('LAP-PRO-16', 'Laptop Workstation Pro 16"', 10, 3500.00)]);

    $this->assertSame(EstadoComprobante::RECHAZADO, $comprobante->estado());
    $this->assertSame(15, $this->productoRepo->buscarPorCodigo('LAP-PRO-16')?->stockActual()); // Stock intacto
}
```

7.4 Test 37: Caso 4 - Flujo Completo Factura → Nota de Crédito por Devolución

```
#[TestDox('Caso 4 (Flujo Completo): Emite Factura -> Emite Nota de Crédito por devolución -> Restaura Stock y actualiza saldo')]
public function testFlujoCompletoFacturaYNotaDeCreditoRestauraStockEnKardex(): void
{
    $sunatMock = $this->createMock(SunatGatewayInterface::class);
    $sunatMock->expects($this->exactly(2))->method('enviarComprobante')->willReturn(new RespuestaSunatCdr('0', 'Aceptado por SUNA
T'));

    $emitirFactura = new EmitirComprobanteUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xmlGene
rator, $this->digitalSigner, $sunatMock);
    $emitirNotaCredito = new EmitirNotaCreditoUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xml
Generator, $this->digitalSigner, $sunatMock);

    // 1. Factura por 4 unidades (Stock baja de 15 a 11)
    $factura = $emitirFactura->ejecutar(serie: 'F001', tipo: TipoComprobante::FACTURA, cliente: Cliente::conRuc('20100047218', 'B
ANCO DE CREDITO'), items: [new ItemComprobante('LAP-PRO-16', 'Laptop', 4, 3500.00)]);
    $this->assertSame(11, $this->productoRepo->buscarPorCodigo('LAP-PRO-16')?->stockActual());

    // 2. Nota de Crédito devolviendo 1 unidad
    $notaCredito = $emitirNotaCredito->ejecutar(serieNota: 'FC01', numeroComprobanteOrigen: $factura->numeroCompleto(), motivo: M
otivoNotaCredito::DEVOLUCION_POR_ITEM, sustento: 'Devolución', itemsDevolucion: [new ItemComprobante('LAP-PRO-16', 'Laptop', 1, 3
500.00)]);
    $this->assertSame('FC01-00000001', $notaCredito->numeroCompleto());

    // 3. El stock sube automáticamente a 12 y el saldo de la factura baja en S/ 4,130.00
    $this->assertSame(12, $this->productoRepo->buscarPorCodigo('LAP-PRO-16')?->stockActual());
    $facturaActualizada = $this->comprobanteRepo->buscarPorNumero('F001-00000001');
    $this->assertSame(12390.00, $facturaActualizada?->saldoDisponibleParaNotasCredito());
}
```

7.5 Tests 38 y 39: Comunicación de Baja y Protección Estricta de Inventario

```
#[TestDox('Caso 5 (Comunicación de Baja): Anula una Factura dentro del plazo legal, genera Ticket SUNAT y repone el Stock')]
public function testComunicacionDeBajaAnulaFacturaYReponeStock(): void
{
    $sunatMock = $this->createMock(SunatGatewayInterface::class);
    $sunatMock->method('enviarComprobante')->willReturn(new RespuestaSunatCdr('0', 'Aceptado'));
    $sunatMock->expects($this->once())->method('comunicarBaja')->willReturn('2026009876543');

    $emitirFactura = new EmitirComprobanteUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xmlGene
rator, $this->digitalSigner, $sunatMock);
    $anularBaja = new AnularComprobanteBajaUseCase($this->comprobanteRepo, $this->productoRepo, $sunatMock);

    // Emite factura (Stock baja de 40 a 35)
    $factura = $emitirFactura->ejecutar('F001', TipoComprobante::FACTURA, Cliente::conRuc('20100047218', 'BCP'), [new ItemComprob
ante('TEC-MEC-RGB', 'Teclado', 5, 250.00)], fechaEmision: new DateTimeImmutable('2026-11-10 09:00:00'));
    $this->assertSame(35, $this->productoRepo->buscarPorCodigo('TEC-MEC-RGB')?->stockActual());

    // Da de baja y repone stock a 40
    $comprobanteAnulado = $anularBaja->ejecutar($factura->numeroCompleto(), 'Duplicidad', new DateTimeImmutable('2026-11-11 11:0
0:00'));
    $this->assertSame(EstadoComprobante::ANULADO_BAJA, $comprobanteAnulado->estado());
    $this->assertSame(40, $this->productoRepo->buscarPorCodigo('TEC-MEC-RGB')?->stockActual());
}

#[TestDox('Caso 6 (Protección de Inventario): Impide emitir un comprobante y no llama a SUNAT si no hay stock suficiente')]
public function testImpideEmitirComprobanteYNoLlamaASunatSiNoHayStock(): void
{
    $sunatMock = $this->createMock(SunatGatewayInterface::class);
    $sunatMock->expects($this->never())->method('enviarComprobante'); // NUNCA debe invocar a SUNAT

    $useCase = new EmitirComprobanteUseCase($this->comprobanteRepo, $this->productoRepo, $this->calculadora, $this->xmlGenerator,
$this->digitalSigner, $sunatMock);

    $this->expectException(StockInsuficienteException::class);

    // Stock disponible: 15; intento de venta: 50
    $useCase->ejecutar('F001', TipoComprobante::FACTURA, Cliente::conRuc('20100047218', 'BCP'), [new ItemComprobante('LAP-PRO-1
6', 'Laptop', 50, 3500.00)]);
}
```

8. Certificación de Ejecución en Terminal

La verificación de toda la suite se realiza de manera autónoma en la terminal CLI mediante el binario oficial de PHPUnit:

```
cd Billing/backend
vendor/bin/phpunit --testdox --colors=never
```

Salida Oficial del Ejecutor de Pruebas:

```
PHPUnit 10.5.66 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.13
Configuration: D:\Escritorio\redobia-web\Billing\backend\phpunit.xml

.....................................                             39 / 39 (100%)

Time: 00:00.216, Memory: 10.00 MB

Suite Feature (Casos de Uso con Mocks y Spies): Simulación de SUNAT, Rollback de Kardex y Ciclo de Vida
 [x] Caso 1 (Mock Aceptado): Emite Factura, SUNAT devuelve CDR código 0 y se descuenta el stock en el Kardex
 [x] Caso 2 (Mock Caída de Red / Timeout): Si SUNAT falla por red, la factura pasa a CONTINGENCIA sin perder la venta
 [x] Caso 3 (Mock Rechazo Fiscal SUNAT): Si SUNAT rechaza la factura (código 2800), hace Rollback automático del Stock
 [x] Caso 4 (Flujo Completo): Emite Factura -> Emite Nota de Crédito por devolución -> Restaura Stock y actualiza saldo
 [x] Caso 5 (Comunicación de Baja): Anula una Factura dentro del plazo legal, genera Ticket SUNAT y repone el Stock
 [x] Caso 6 (Protección de Inventario): Impide emitir un comprobante y no llama a SUNAT si no hay stock suficiente

Suite Unitaria 1: Validación de Value Objects Tributarios (RUC Módulo 11, DNI, Series y Monedas)
 [x] Acepta números de RUC válidos verificando el algoritmo oficial Módulo 11 de SUNAT (4 datasets)
 [x] Rechaza números de RUC con longitud incorrecta, prefijo no autorizado o dígito verificador alterado (6 datasets)
 [x] Valida DNI de 8 dígitos y rechaza DNI de puros ceros o alfanumérico
 [x] Formatea correctamente Serie y Correlativo a 8 dígitos (F001-00000045)
 [x] Impide usar una serie de Boleta (B001) en una Factura Electrónica (01)
 [x] Exige un tipo de cambio válido mayor a 1.00 cuando la moneda es Dólares (USD)

Suite Unitaria 2: Motor de Cálculo Tributario (IGV 18%, Exonerados, Gratuitos, ICBPER, Descuentos y Detracciones)
 [x] Calcula exactamente una operación gravada estándar con IGV 18%
 [x] Calcula factura mixta: Gravados + Exonerados + Inafectos + Bonificación Gratuita + Bolsas ICBPER
 [x] Aplica Detracción SPOT (12%) automáticamente cuando el servicio supera S/ 700.00 o equivalente en USD
 [x] No aplica Detracción SPOT si el importe total en Soles no supera los S/ 700.00
 [x] Lanza excepción si se intenta aplicar descuento comercial a una bonificación gratuita
 [x] Calcula exactamente una operación inafecta al IGV (30) con base imponible y 0% de IGV

Suite Unitaria 3: Reglas de Negocio del Comprobante, Notas de Crédito y Comunicaciones de Baja
 [x] Regla SUNAT 2017: Impide emitir una Factura Electrónica (01) a un cliente con DNI
 [x] Regla SUNAT 2511: Impide emitir Boleta mayor a S/ 700.00 a Clientes Varios (Sin DNI)
 [x] Permite emitir Boleta menor o igual a S/ 700.00 a Clientes Varios
 [x] Controla el saldo de una Factura ante Notas de Crédito parciales e impide devolver más del total
 [x] Impide anular por Comunicación de Baja una Factura emitida hace más de 7 días calendario
 [x] Permite Comunicación de Baja dentro de los 7 días y cambia estado a ANULADO_BAJA

Suite de Integración 1: Estándar XML OASIS UBL 2.1, Firma Criptográfica SHA-256 y Detección de Adulteración
 [x] Genera un documento XML UBL 2.1 válido y verifica sus nodos obligatorios mediante consultas XPath
 [x] Firma digitalmente el XML UBL 2.1 con SHA-256 e inyecta DigestValue y SignatureValue verificables
 [x] Detecta adulteración (ataque de modificación de importe en el XML firmado) y SUNAT lo rechaza con código 2335
 [x] Genera XML UBL 2.1 con operaciones inafectas (30) asociadas al esquema tributario SUNAT 9998 INA / FRE

Suite de Integración 2: Persistencia Relacional SQLite (:memory:), Correlativos Secuenciales y Kardex
 [x] Sembrado inicial carga el catálogo de productos y permite actualizar el stock en SQLite
 [x] Persiste y recupera un Comprobante completo manteniendo su correlativo secuencial por serie
 [x] La tabla de Idempotencia registra transacciones IN_PROGRESS y recupera la respuesta COMPLETED

OK (39 tests, 140 assertions)
```